



CSE461

Section : 04

Group : 04

Semester : Spring 2025

Project Title:

Autonomous Aquarium Water Monitoring, fish feeding and aerator system.

Member Details:

1. Name: Ameet Faisal	ID: 22201869
2. Name: Shahriar	ID: 22201898
3. Name: Khan Farhan Mahdi	ID: 22201698
4. Name: Shahriar Mohammad	ID: 22299020
5. Name: Md Sahin Siraj	ID: 18201104

Project Demonstration Date: 10 May 2025

Instructors: Pratick Roy Chowdhuruy &

Maharin Mosfakin

Table of Contents

Table of Contents.....	1
Components.....	1
1. Code Documentation.....	2
2. User Manual.....	5
Adding Connections -.....	5
Sensor Connections:.....	5
Actuator Connections:.....	5
Power:.....	6
Software install and Code run -.....	6
Step 1: Install Arduino IDE.....	6
Step 2: Install Required Libraries.....	6
Step 3: Upload Code.....	6
Operation Instructions -.....	7
Step-by-Step Usage -.....	7
Startup.....	7
Sensor Monitoring -.....	7
Motor & Servo Control -.....	7
Possible troubleshooting -.....	8
Calibration.....	9
pH Sensor calibration -.....	9
To calibrate:.....	9
Safety and Maintenance.....	11
3. Testing and Modifications Log.....	13
4. Supporting Materials.....	14
Schematic Diagram.....	14
Flowcharts.....	16
Citations of components and Resources.....	18
Components:.....	18
5. Demonstration Video.....	19

Components

- **Microcontroller:** Arduino Uno
- **Sensors:**
 - pH Sensor
 - Temperature Sensor
 - Turbidity Sensor
- **Actuators:**
 - Water Pump
 - Servo Motor
 - Relay Module
- **Body/Chassis:** Hardboard and Breadboards
- **Additional Components:** Battery, Resistors and Wires

1. Code Documentation

For Motor:

```
include <Servo.h> // Include the Servo library to control servo motors

// Define pins for DC motor control
int motor1pin1 = 2; // Control pin 1 for DC motor (connected to H-bridge)
int motor1pin2 = 3; // Control pin 2 for DC motor (connected to H-bridge)

Servo myServo; // Create a Servo object to control the servo motor

void setup() {
  // Setup for DC motor pins as outputs
  pinMode(motor1pin1, OUTPUT); // Set motor1pin1 as output
  pinMode(motor1pin2, OUTPUT); // Set motor1pin2 as output

  // Setup for Servo motor
  myServo.attach(9); // Attach the servo's signal wire to digital pin 9
}

void loop() {
  // DC Motor control - runs forward continuously
  digitalWrite(motor1pin1, LOW); // Set pin1 LOW
  digitalWrite(motor1pin2, HIGH); // Set pin2 HIGH to run motor forward

  // Servo motor movement sequence:
  // 1. Move servo to 0 degree position
  myServo.write(0); // Command servo to move to 0 degrees
  delay(10000); // Wait for 10 seconds in this position

  // 2. Move servo to 160 degree position
  myServo.write(160); // Command servo to move to 160 degrees
  delay(1000); // Wait for 1 second in this position

  // The loop will then repeat indefinitely:
  // - DC motor keeps running forward continuously
  // - Servo cycles between 0° (10 sec) and 160° (1 sec)
}
```

For Sensor:

```
#include <OneWire.h>           // Library for 1-wire communication protocol
#include <DallasTemperature.h>  // Library for Dallas temperature sensors

// --- Temperature Sensor Setup ---
#define ONE_WIRE_BUS 2         // Digital pin for temperature sensor data
OneWire oneWire(ONE_WIRE_BUS); // Setup oneWire instance
DallasTemperature sensors(&oneWire); // Pass oneWire reference to Dallas Temp
DeviceAddress insideThermometer; // Array to hold device address

// --- Turbidity Sensor Setup ---
const int turbidityPin = A0;    // Analog pin for turbidity sensor
const int ledPin = 7;           // Digital pin for warning LED

// --- pH Sensor Setup ---
const int pHPin = A1;           // Analog pin for pH sensor
int pH_Value;                   // Store raw pH sensor reading
float Voltage;                  // Store converted voltage value

void setup() {
  Serial.begin(9600);           // Initialize serial communication

  // Set sensor pins as inputs and LED as output
  pinMode(turbidityPin, INPUT);
  pinMode(pHPin, INPUT);
  pinMode(ledPin, OUTPUT);

  // Initialize temperature sensor
  Serial.println("Initializing Dallas Temperature Sensor...");
  sensors.begin();              // Start up the temperature sensor library
  Serial.print("Found ");
  Serial.print(sensors.getDeviceCount(), DEC);
  Serial.println(" device(s).");

  // Try to locate temperature sensor device
  if (!sensors.getAddress(insideThermometer, 0))
    Serial.println("Unable to find address for Device 0");
  else {
    // Print device address if found
    Serial.print("Device 0 Address: ");
    printAddress(insideThermometer);
    Serial.println();

    // Set resolution to 9 bits (0.5°C precision)
    sensors.setResolution(insideThermometer, 9);
    Serial.print("Device 0 Resolution: ");
    Serial.println(sensors.getResolution(insideThermometer), DEC);
  }
}
```

```

}

void loop() {
  // --- Turbidity Measurement ---
  int turbidityValue = analogRead(turbidityPin); // Read raw turbidity (0-1023)
  Serial.print("Turbidity Sensor Value: ");
  Serial.println(turbidityValue);

  // Turbidity warning system (threshold at 500)
  if (turbidityValue < 500) { // Lower value means more turbid
    digitalWrite(ledPin, HIGH); // Turn on warning LED
    Serial.println("Warning: High Turbidity Detected!");
  } else {
    digitalWrite(ledPin, LOW); // Turn off LED if water is clear
  }

  // --- Temperature Measurement ---
  sensors.requestTemperatures(); // Command all devices to measure temp
  float tempC = sensors.getTempC(insideThermometer); // Get temp in Celsius

  // Check if sensor reading is valid
  if (tempC == DEVICE_DISCONNECTED_C) {
    Serial.println("Error: Could not read temperature data");
  } else {
    // Print temperature in both Celsius and Fahrenheit
    Serial.print("Temperature: ");
    Serial.print(tempC);
    Serial.print(" °C / ");
    Serial.print(DallasTemperature::toFahrenheit(tempC)); // Convert to F
    Serial.println(" °F");
  }

  // --- pH Measurement ---
  pH_Value = analogRead(pHPin); // Read raw pH sensor value (0-1023)
  Voltage = pH_Value * (5.0 / 1023.0); // Convert to voltage (0-5V)
  Serial.print("pH Sensor Voltage: ");
  Serial.println(Voltage); // Note: Needs calibration to actual pH

  Serial.println("-----");
  delay(2000); // Wait 2 seconds between readings
}

// Helper function to print device address
void printAddress(DeviceAddress deviceAddress) {
  for (uint8_t i = 0; i < 8; i++) {
    if (deviceAddress[i] < 16) Serial.print("0"); // Pad single hex digits
    Serial.print(deviceAddress[i], HEX); // Print address byte
  }
}

```

2. User Manual

Adding Connections -

Sensor Connections:

Temperature Sensor (DS18B20):

- Data pin → Digital Pin 2 (with 4.7k Ω pull-up to 5V)
- VCC → 5V
- GND → GND

Turbidity Sensor:

- Output pin → A0
- VCC → 5V
- GND → GND

pH Sensor:

- Analog output → A1
- VCC → 5V
- GND → GND

Actuator Connections:

DC Motor (Water Pump):

- Motor pins → Digital Pins 2 and 3 (via H-Bridge)
- External power for motor (e.g., 9V battery)

Servo Motor:

- Signal → Digital Pin 9
- VCC → 5V
- GND → GND

Relay Module:

- 12v → From battery Positive source
- GND → Arduino GND and Battery GND
- 5v → Arduino 5v
- Out1 → Pump Positive
- Out2 → Pump Negative

Power:

Connect Arduino to Laptop via USB.

Software install and Code run -**Step 1: Install Arduino IDE**

Download and install from: <https://www.arduino.cc/en/software>

Step 2: Install Required Libraries

In Arduino IDE:

Go to Sketch > Include Library > Manage Libraries...

Search and install:

- OneWire
- DallasTemperature
- Servo

Step 3: Upload Code

- Open your .ino sketch with the provided code (Sensors and Motors).
- Connect Arduino via USB.
- Select board: Tools > Board > Arduino Uno
- Select COM Port: Tools > Port > (Select correct COM)
- Click Upload.

Operation Instructions -

Step-by-Step Usage -

Startup

- Ensure all components are connected properly.
- Power on the Arduino via USB or battery.
- Open Serial Monitor (9600 baud) to see real-time sensor data.

Sensor Monitoring -

- Temperature: Printed in °C and °F.
- pH Sensor: Displays voltage (calibration required for exact pH).
- Turbidity: If value is < 500, the LED will light up as a warning.

Motor & Servo Control -

- DC Motor: Runs continuously in one direction.
- Servo Motor: Rotates from 0° → waits 10s → rotates to 160° → waits 1s → repeats.
- Relay (Pump): Can be controlled manually with added code depending on conditions.

Possible troubleshooting -

Issue	Cause	Solution
No Serial Output	COM port not selected / Baud rate mismatch	Check Tools > Port and set baud rate to 9600
Servo not moving	Not connected to 5V / PWM pin wrong	Ensure connected to D9 and has proper voltage
DC motor not running	No H-bridge or incorrect wiring	Check H-bridge wiring, ensure motor1pin1 is LOW, pin2 is HIGH
Temperature shows -127°C	Sensor not detected	Check DS18B20 wiring and pull-up resistor (4.7kΩ between data and VCC)
LED not lighting for turbidity	Sensor always returns high values	Check sensor placement, use clean vs. dirty water for testing
pH readings seem incorrect	Sensor uncalibrated	Use known pH solutions to map voltage to accurate pH
Arduino resets randomly	Power instability	Use a proper power source or capacitor smoothing

Calibration

The sensor comes calibrated but there might be a need for calibration.

pH Sensor calibration -

To calibrate:

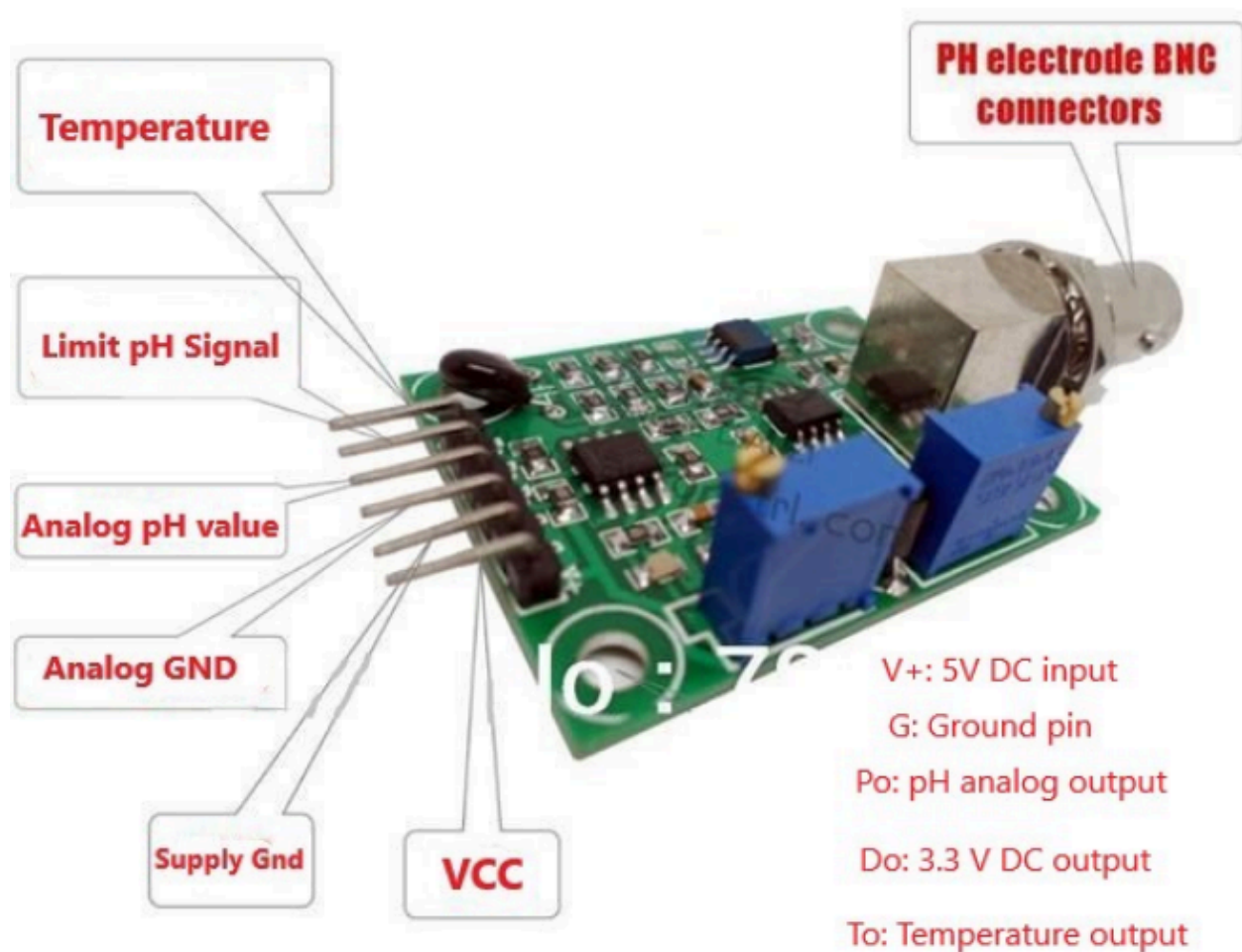
- Use buffer solutions of known pH (e.g., 4.0, 7.0, 10.0).
- Record the voltage for each.
- Use linear mapping in code to convert voltage → pH.

Calibration Code -

```
#include <Arduino.h>
const int adcPin = A0;

void setup()
{
  Serial.begin(115200);
}

void loop()
{
  int adcValue = analogRead(adcPin);
  float phVoltage = (float)adcValue * 5.0 / 1024;
  Serial.print("ADC = ");
  Serial.print(adcValue);
  Serial.print(" Po = ");
  Serial.println(phVoltage, 3);
  delay(1000);
}
```



We can see that there are two potentiometers in the above pic. Which is closer to the BNC connector of the probe is the offset regulation, the other is the pH limit.

Offset: The average range of the probe oscillates between negative and positive values. The 0 represents a pH of 7.0. In order to be able to use it with Arduino, this circuit adds an offset value to the value measured by the probe, so the ADC will only have to take samples of positive voltage values.

PH Limit: This potentiometer is to set a limit value of the pH sensor circuit that causes the red LED to light up and the Do pin signal to turn ON.

Connect Po to Analog input A0 on Arduino, and G to Arduino GND. Run the given Arduino sketch above, and open the Serial Monitor of Arduino IDE to observe the reading, slowly adjust the potentiometer RV1 (the one near the BNC connector on the board) until the Po reading is equal to 2.50v.

Safety and Maintenance

- Always power off the system before changing connections.
- Avoid water contact with Arduino or electronics.
- Clean sensors regularly to prevent buildup.
- Use gloves when handling water of unknown quality.

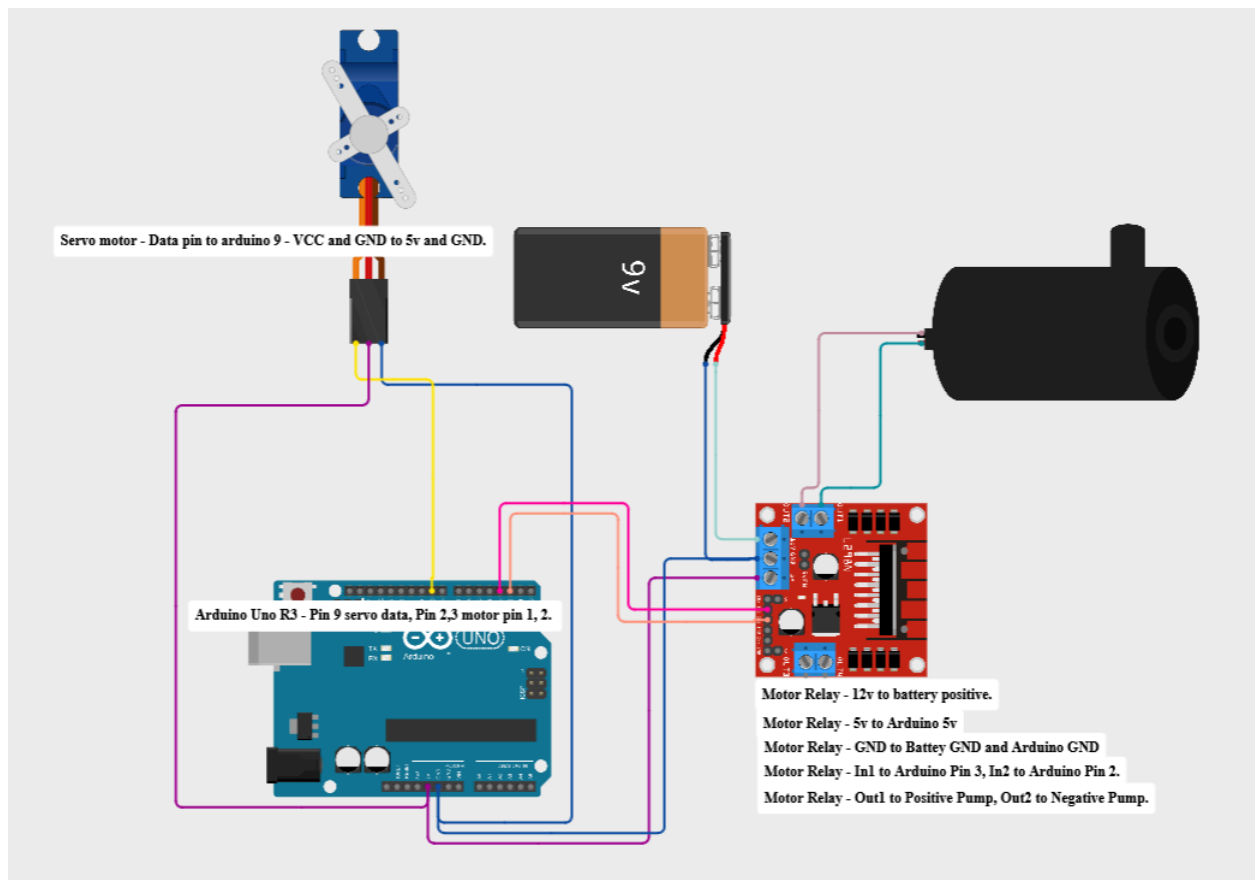


Figure 1 - Arduino and Motors

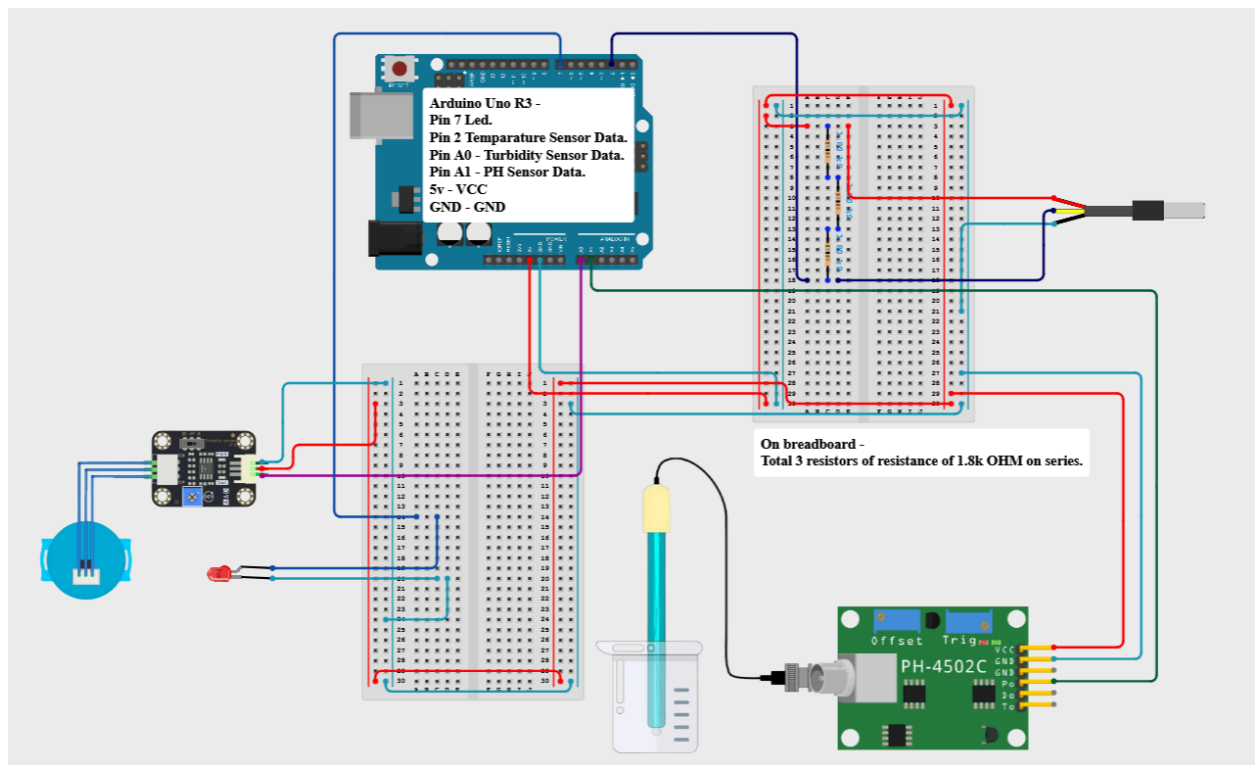


Figure 2 - Arduino and Sensors.

3. Testing and Modifications Log

Hardware Implementation & Challenges

1. **Implemented Most Hardware** – Successfully integrated the majority of the initially planned hardware components.
2. **Excluded Some Components** – Decided not to use the following due to budget constraints:
 - Water Level Sensor
 - Peristaltic Pump
 - Air Pump
 - Solenoid Valves
3. **Testing Obstacles Faced** – Encountered several issues during hardware testing:
 - Faulty Resistance – Some resistors in the circuit were defective.
 - Temperature Sensor Malfunction – Initially gave incorrect readings (around -200°C).
 - pH Sensor Issue – Provided a constant value (between 3-4 pH) in all conditions (clean water & lemon water).
4. **Solutions Applied**
 - Replaced faulty resistors with new ones.
 - Purchased a new temperature sensor, which then provided accurate readings.
5. **Unresolved Issue** – The pH sensor problem persisted:
 - Initially gave some readings, but after connecting to its board, it only showed 3-4 pH regardless of the liquid tested.
 - Could not replace it due to budget constraints (it was the most expensive component).

4. Supporting Materials

Schematic Diagram

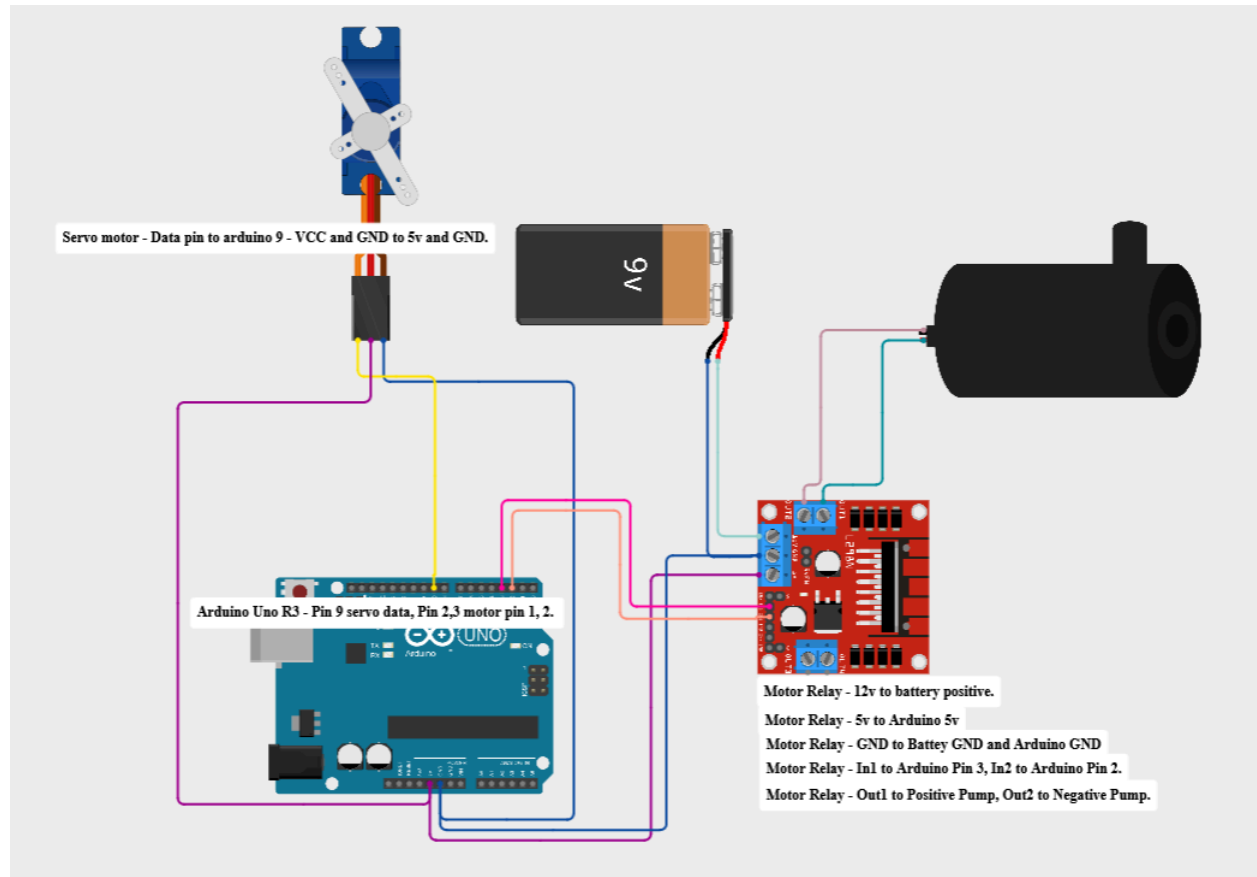


Figure 1 - Arduino and Motors

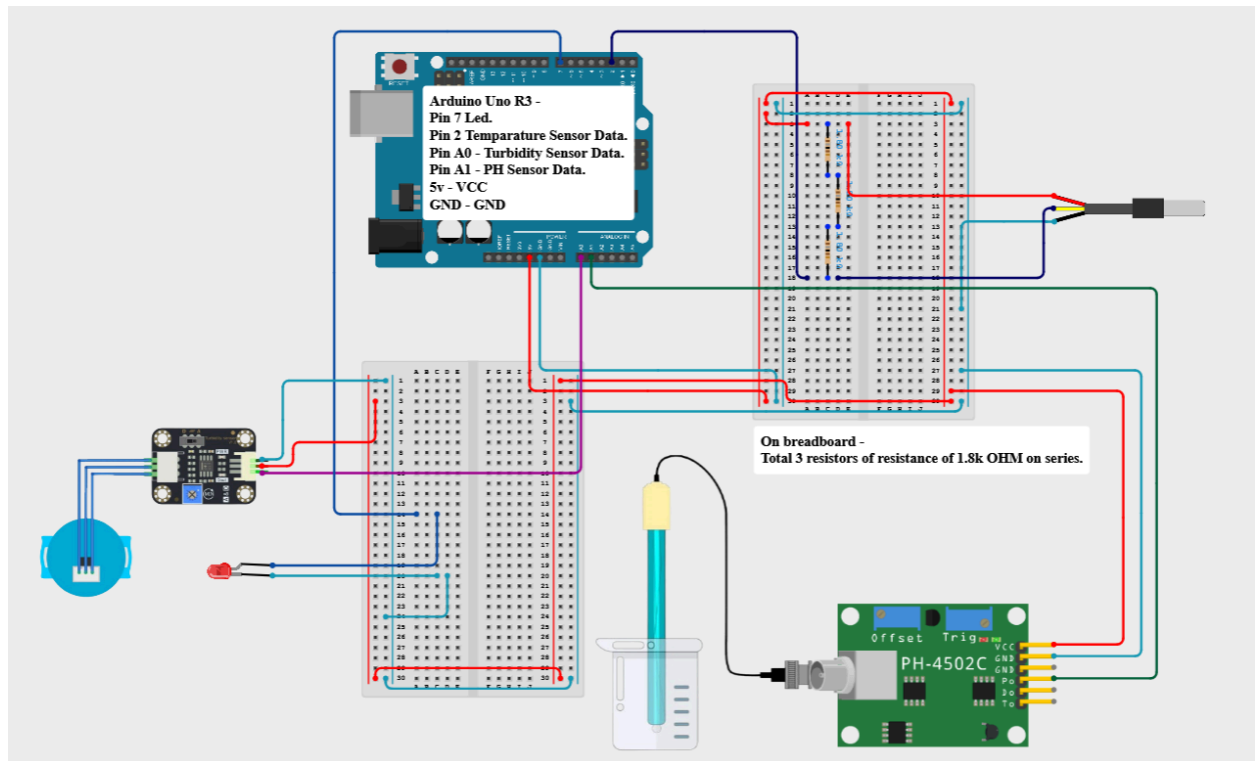
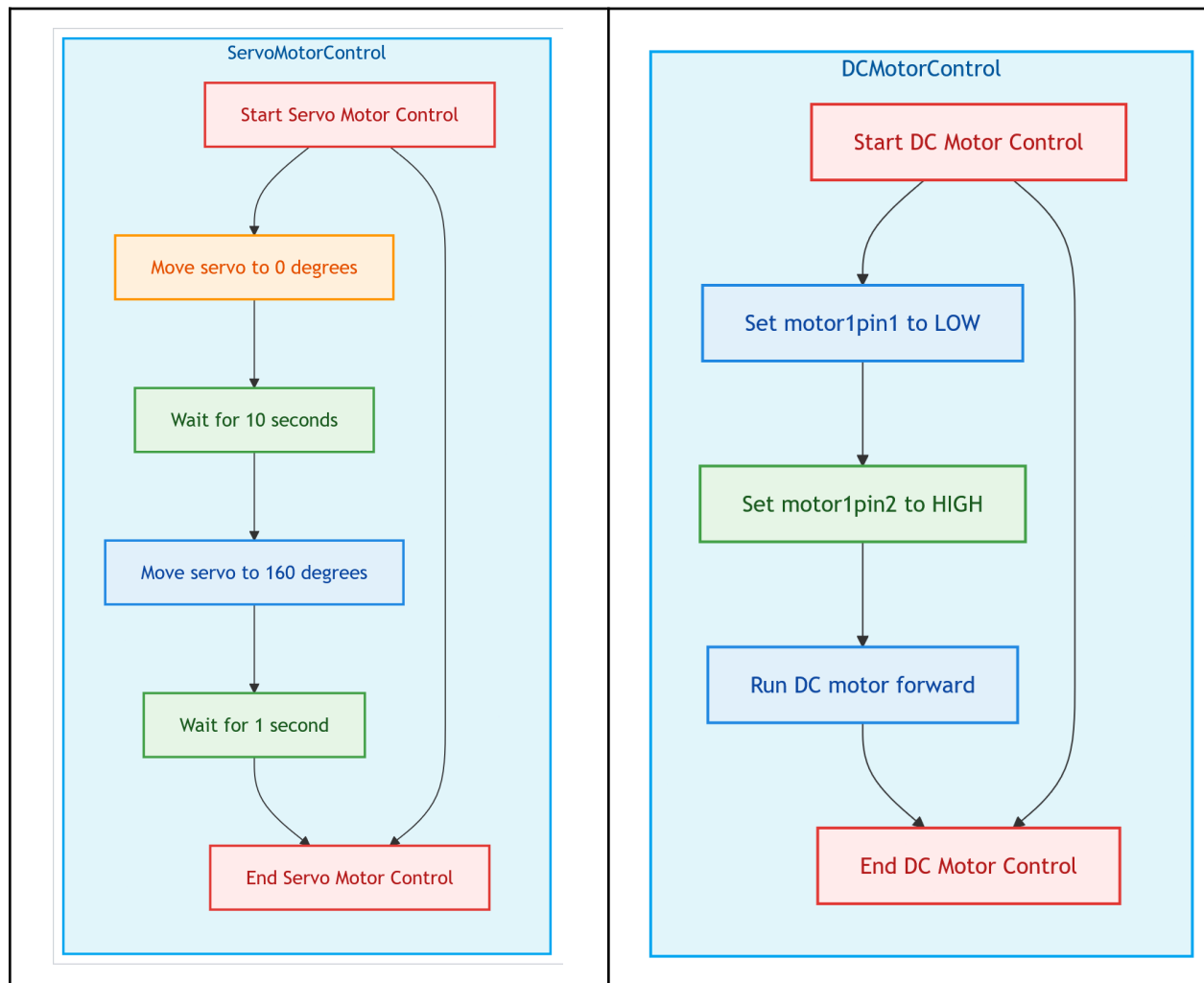


Figure 2 - Arduino and Sensors.

Flowcharts



Arduino Water Quality Monitoring Flow Chart



Citations of components and Resources

Components:

1. pH Sensor

Resources: <https://tutorials.probots.co.in/calibrating-a-ph-sensor/>
<https://www.supmeaauto.com/uploads/2101/ph-sensor-datasheet.pdf>

2. Temperature Sensor

Resources: <https://arduinogetstarted.com/tutorials/arduino-temperature-sensor>
<https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>

3. Turbidity Sensor

Resources:
<https://circuitdigest.com/microcontroller-projects/measuring-turbidity-of-water-to-determine-water-quality-using-arduino-turbidity-sensor>
https://www.aanderaa.com/media/pdfs/d424_turbidity_sensor_4296.pdf

4. Servo Motors

Resources: <https://www.instructables.com/Arduino-Servo-Motors/>
http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf

5. L298n Motor Driver

Resources:
<https://projecthub.arduino.cc/lakshyajhalani56/l298n-motor-driver-arduino-motors-motor-driver-l298n-7e1b3b>
<https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>

6. Library for Servo Motor

Resources: <https://docs.arduino.cc/libraries/servo/>

7. Temperature Control Library

Resources: <https://github.com/milesburton/Arduino-Temperature-Control-Library>

8. Arduino Uno

Resources: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

5. Demonstration Video

Project Demonstration Video:

https://youtu.be/BNTS_jkfV6w

https://www.youtube.com/shorts/MrS_cD5rP-0